

Mizar Evo

Detailed discussion deck for the Białystok Mizar team

Mizar Evo project

September 2026

Title:

Mizar Evo

Readable, AI-Ready, Scalable Formal Mathematics

Subtitle:

Eight problems, eight proposals

A discussion with the Bialystok Mizar team, September 2026

Legacy Mizar (exact MML excerpt):

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

Mizar Evo (specification example):

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;    :: renamed view
  end;
end;
```

Key Phrase:

Preserve Mizar's mathematical vernacular.

Modernize the compiler, verifier, artifact, and publication layers.

What this talk does not claim:

- Full MML migration is not complete.
- The final language standard is not frozen.
- AI assistance is not a substitute for proof checking.
- The current Mizar system's achievements are the baseline, not the problem.

Every code example is labeled:

Label	Meaning
exact MML excerpt	verbatim current MML text, with article and line numbers
specification example	from the Mizar Evo language specification
sketch	illustrative; not yet fixed by the specification

Reading rule:

- No EBNF appears in this talk. The language is shown through examples only; doc/spec/en/ remains the authority for grammar and edge cases.

Key Points:

- identify compatibility constraints that are invisible from outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;
- discuss how Formalized Mathematics should link to a package library;
- collect the objections we have not thought of.

Running question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Key Points:

- declarative proof text that reads as mathematics;
- soft types, modes, and adjective-rich vocabulary;
- attributes, registrations, and clusters as reusable automation;
- a mature curated library (MML 5.94.1493: 1493 articles);
- a publication culture around formal articles (Formalized Mathematics).

Message:

- These strengths are the design baseline. Every proposal in this talk is judged by whether it protects them.

Key Points:

- MML has grown to roughly 1500 interdependent articles.
- The unit of dependency, review, and reuse is the whole article.
- Article environments are resolved by tooling, but the resolved dependency surface is not visible in the source a human reviews.
- Whole-library maintenance operations (renames, refactorings, revisions) carry risk that grows with library size.

Message:

- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

Key Points:

- Editors are expected to give instant, partial, resilient feedback.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work at package granularity with versioning.
- Documentation is expected to be generated, linked, and browsable.

Message:

- These expectations were set by mainstream language ecosystems; new users arrive with them, and formal libraries are judged against them.

Key Points:

- AI agents are already useful for search, explanation, and repair.
- They need bounded, structured, source-anchored context - not a dump of the whole library.
- Their output must never define proof truth; verification must stay independent of the strength of the assistant.
- Readable source is an advantage here: stable local text patterns are what AI edits and retrieval work best on.

Message:

- Mizar's readability is not a nostalgic asset. It is exactly what makes safe AI assistance possible.

Key Phrase:

Do not trade away readability to gain automation.
Use automation to protect and extend readability.

Three pillars, one test:

Pillar	Test for every feature
Readability	does proof text still read as mathematics?
AI-readiness	can a tool see bounded, auditable context?
Scalability	do boundaries stay stable as the library grows?

Legacy Mizar (exact MML excerpt):

```
environ

  vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,
    FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;
  notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,
    CARD_1, STRUCT_0;
  constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;
  registrations ZFMISC_1, CARD_1, STRUCT_0;
  theorems STRUCT_0;

begin :: Additive structures
```

Key Points:

- One symbol's origin is spread over several role lists; a reviewer cannot see which article contributes which notation, constructor, or cluster.
- The Accommodator resolves the environment, but the resolved surface is not reviewable source text.
- Tools cannot cache or invalidate at a finer granularity than the article.
- Moving a theorem between articles risks breaking unknown dependents.

Message:

- Implicit dependency surfaces are a fixed cost on every edit, every review, and every tool, and the cost grows with the library.

Mizar Evo (specification example):

```
import .function;  
import mml.algebra.structure.sorted;  
  
definition  
  let S be 1-sorted;  
  mode BinOpDef: BinOp of S is  
    Function of [: S.carrier, S.carrier :], S.carrier;  
end;
```

Rules that make this deterministic:

- all imports appear before the first item; no mid-file environment changes;
- imports seed the active lexicon, and every symbol, notation, registration, and theorem is traceable to exactly one import;
- stable fully-qualified names are derived from package and module paths.

Mizar Evo (specification example):

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

Key Points:

- a manifest plus a lockfile makes every build reproducible;
- versioned reuse (SemVer) replaces ad hoc copying between article sets.

Migration map:

Current environ role	Evo target
vocabularies	exported symbols and lexical metadata
notations	imported notation metadata
constructors	visible definitions and constructors
registrations	import-scoped registration index
requirements	package or prelude policy
article theorem labels	module-qualified theorem identities

Message:

- this is not a mechanical rename: current environments mix semantic, syntactic, and automation-facing roles, and migration reports must explain what each imported module actually contributes.

Preserved:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).

Questions for Bialystok:

- Which `environ` roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

Legacy Mizar (exact MML excerpt):

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

Key Points:

- parent link, fields, and selector layout are one compact declaration;
- with multiple parents, the merge of inherited fields is implicit;
- renamed views (additive vs multiplicative) rest on naming conventions;
- stored data and canonical values (a zero, a unit) are not distinguished.

Key Points:

- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- A migration tool cannot ask "is this selector intrinsic data, a canonical value with obligations, or an inherited view?" - the syntax does not say.
- Errors surface far from their cause, as type mismatches in later articles.

Message:

- At MML scale, structure inheritance is a graph maintenance problem, and the graph deserves explicit, checkable edges.

Mizar Evo (specification example):

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

Three distinct concepts:

Concept	Meaning	Consequence
field	intrinsic data supplied by the value	constructor shape, equality
property	uniquely determined canonical value	existence/uniqueness obligations
attribute	predicate-style refinement	cluster propagation, not layout

Mizar Evo (specification example):

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;      :: renamed
  end;
end;
```

Key Points:

- one inherit statement per parent; mapping and renaming are source text;
- narrowing and type conversions carry coherence proof obligations;
- the additive/multiplicative naming convention becomes a checked view.

Mizar Evo (specification example):

```
struct DoubleLoopStr where
  field carrier -> set;
  field add -> BinOp of carrier;
  field mul -> BinOp of carrier;
  property zero -> Element of carrier;
  property one -> Element of carrier;
end;

inherit DoubleLoopStr extends AddLoopStr;
inherit DoubleLoopStr extends MulLoopStr;
```

Message:

- The analyzer must check that both inheritance paths introduce the same components: `add -> LoopStr.binop -> Magma.binop` along path one must agree with `add -> AddMagma.add -> Magma.binop` along path two.
- Diamond inheritance becomes a diagnostic with source spans, not a silent merge decided by declaration order.

Preserved:

- structures remain Mizar structures: carriers, selectors, `Element of`;
- aggregate compactness is traded only where a hidden decision existed.

Questions for Bialystok:

- Is the `field / property / attribute` split readable in real algebraic articles, or does it over-annotate simple cases?
- Is `one-parent-per-inherit` acceptable for the inheritance-heavy parts of MML, such as the `ALGSTR` and topology hierarchies?
- Which MML structures would be the best diamond test cases?

Legacy Mizar (exact MML excerpt):

```
registration
  let M be addMagma;
  cluster right_add-cancelable left_add-cancelable -> add-cancelable for
Element
  of M;
  coherence;
end;
```


Key Points:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer; the cause lives somewhere in the environment.
- Which registrations fired, in which order, is invisible; the automation is powerful, but its explanation does not scale with its power.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Message:

- Automation that cannot explain itself becomes a maintenance liability at library scale - even when it is sound.

Mizar Evo (specification example):

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Key Points:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier stores an import-filtered cluster resolution graph;
- @show_resolution and explanation artifacts answer "why (not)?" with the actual chain, e.g. empty -> finite -> countable.

Mizar Evo (specification example):

```
registration
  let n be Nat;
  reduce NatAddZero:  $n + 0$  to  $n$ ;
  reducibility
  proof
    let n be Nat;
    thus  $n + 0 = n$  by mml.number.natural.Nat_add_zero;
  end;
end;
```

Key Points:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (specificity first, FQN tie-break);
- unoriented identification idioms become auditable reduce items.

Preserved:

- registrations and clusters remain first-class; proofs stay short;
- no new proof text is required at use sites - only traces are added.

Questions for Bialystok:

- Which cluster explanations would most reduce daily friction: failure explanations, firing traces, or difference reports between environments?
- Which MML article families stress registrations hardest and should become migration benchmarks for the cluster graph?

Key Points:

- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.

Key Phrase:

How do we get modern proof search
without trusting the searcher?

Mizar-side semantics

-> well-typed, resolved obligations

ATP-side search

-> candidate proof evidence

kernel-side checking

-> accepted or rejected proof status

Key rules:

- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel never searches for proofs;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

Certificate

- target VC fingerprint
- kernel profile
- imported facts and hashes
- generated clauses
- substitutions
- resolution trace
- final goal

Key Points:

- accepted evidence is normalized into replayable certificate data;
- the kernel checks imported facts, substitutions, clause well-formedness, and the resolution/SAT trace - it does not trust a solver's exit code, so unsoundness in search cannot become unsoundness in accepted results;
- a proof accepted under one dependency slice cannot silently migrate to another: the hashes pin it down.

Legacy Mizar (exact MML excerpt):

```
theorem
  for F being non degenerated ZeroOneStr holds 1.F in NonZero F
proof
  let F be non degenerated ZeroOneStr;
  not 1.F in {0.F} by TARSKI:def 1;
  hence thesis by XBOOLE_0:def 5;
end;
```

Message:

- Citation repair - proposing a missing or sharper by reference - is the canonical safe AI edit: source-local, meaning-preserving, and checked by the verifier like any human edit.

5.5 - Edit Classes: Green, Yellow, Red

Class	Examples	Policy
Green	add citation, insert qua, info annotation	auto-proposable, still verified
Yellow	add import, local lemma, registration	proposed with human review
Red	weaken theorem, change definition, add axiom	forbidden to ordinary agents

Forbidden repair (sketch):

```
theorem
  for x be Nat holds  $x + 0 = x$  or  $x = 0$ ;
```

Message:

- Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

Preserved:

- the de Bruijn discipline: a small checker judges everything;
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.

Questions for Bialystok:

- Is the SAT/resolution certificate story convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

Key Points:

- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- None of this is a defect of the current design - it is what article-level granularity implies once the library is large.

Key Points:

- hash the source, the dependency interfaces, the generated obligations, and the proof witnesses;
- reuse a result only when fingerprints and verifier policy match;
- a proof-body-only change does not rebuild importers, because public statements and statuses are unchanged;
- independent modules, obligations, ATP runs, and kernel checks run in parallel; results are published in canonical order.

Rule:

Cache reuse is never proof authority.

A clean build must always be able to reproduce every acceptance.

6.3 - The Evo Answer: A Memory Contract

resident memory should scale with:

- active source

- imported public interfaces

- import-filtered indexes

- active module obligations

not with:

- imported proof bodies

- private lemmas outside the interface

- registration data outside the import closure

Message:

- Interfaces are loaded; proof bodies are not. This is what makes whole-MML editing sessions feasible on ordinary hardware.

Preserved:

- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.

Questions for Bialystok:

- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

Legacy Mizar (exact MML excerpt):

```
scheme
  NatInd { P[Nat] } : for k being Nat holds P[k]
provided
A1: P[0] and
A2: for k be Nat st P[k] holds P[k + 1]
```

Key Points:

- schemes carry second-order patterns (induction, separation, replacement) and they work - but they are a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

Key Points:

- `addMagma` and `multMagma` are the same mathematics twice, related only by naming convention (story 2 met this already);
- polynomial rings, vector spaces, and matrix theories are re-spelled per carrier because there is no parameterized construction;
- a theorem proved for one commutative operation is re-proved for `+` and `*` separately.

Message:

- The library pays for the missing generics mechanism in duplicated articles, and every duplicate is a maintenance obligation.

Mizar Evo (specification example):

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

Key Points:

- a template is an ordinary definition block whose leading `let` binds parameters: types, values, predicates, or functors;
- one mechanism covers structures, modes, functors, predicates, theorems, registrations, and algorithms;
- readable shorthands survive: `Module over R` is an automatic synonym for `Module[R]`, `Subset of X` for `Subset[X]`.

Mizar Evo (specification example):

```
definition
  let T be type extends commutative Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
      holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Message:

- `type extends commutative Magma` states what the parameter must provide before any proof search begins;
- the theorem is proved once, for every commutative operation.

Instantiation (specification example):

```
PermProduct[AddMagma]           :: additive form
PermProduct[commutative MulMagma] :: multiplicative form

let R be commutative Ring;
PermProduct[R qua AddMagma]      :: R's additive view
PermProduct[R qua MulMagma]     :: R's multiplicative view
```

Message:

- qua selects the intended view when a ring reaches Magma along two paths - and notation follows the view: the generic * displays as + under the additive instantiation.

Mizar Evo (specification example):

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

Key Points:

- predicate parameters follow the familiar defpred convention;
- legacy schemes have a direct, mechanical migration target;
- instantiation is explicit bracket syntax, so tools and artifacts see exactly which instance a proof uses.

Preserved:

- scheme-style reasoning survives unchanged in power;
- of / over phrasing keeps mathematical prose readable;
- first-order discipline: templates are checked instantiation, not a new logic.

Questions for Bialystok:

- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with of/over as display forms?
- Where should parameter inference stop and demand explicit [T] or qua?

Key Points:

- Current Mizar can define Gcd and prove its theory - but cannot compute `gcd(48, 18)`; every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still serves us.

Key Phrase:

The library describes computation.
It cannot perform or export it.

Mizar Evo (specification example, condensed from spec section 20.12):

```
definition
  let a, b be Nat;
  terminating algorithm euclid_gcd(a, b) -> Nat
    requires a >= 1 & b >= 1
    ensures result = Gcd(a, b)
  do
    var x := a; var y := b;
    while y <> 0 do
      invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);
      decreasing y;
      const r := x mod y; x := y; y := r;
    end;
    return x;
  end;
end;
```

Message:

- ensures and the invariant cite the mathematical Gcd: no circularity.

Mizar Evo (specification example):

```
theorem EuclidGcd12_8:  euclid_gcd(12, 8)  = 4  by computation;  
theorem EuclidGcd100_75: euclid_gcd(100, 75) = 25 by computation;  
  
theorem Fact10: factorial(10) = 3628800  
proof  
  thus thesis by computation(steps: 100000);  
end;
```

Key Points:

- the Mizar Virtual Machine (MVM) evaluates ground goals during verification, under explicit step, time, and depth budgets;
- only ground equalities and ground predicates qualify; everything else still requires a classical proof;
- imported algorithms are opaque: downstream proofs may use only their ensures contract, never their body.

Mizar Evo (specification example):

```
definition
  let n be Nat;
  terminating algorithm factorial(n) -> Nat
    decreasing n
  do
    if n = 0 do return 1; end;
    return n * factorial(n - 1);
  end;
end;
```

Key Points:

- ordinary func definitions are first-order definitional extensions and cannot be recursive;
- a terminating algorithm - once its termination obligations are discharged - is promoted to a genuine functor, usable in any proof;
- this is the only door through which recursion enters the mathematical layer, and it is a proof-shaped door.

Key Points:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by `computation` is MVM replay under budgets, not solver trust;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never feed back into acceptance.

Message:

- Algorithms widen what the library can express; they do not touch what it means for a theorem to be accepted.

Preserved:

- the theorem language is unchanged; algorithms live in `definition` blocks and interact with proofs only through contracts and `verified promotion`;
- first-order set-theoretic foundations stay intact.

Questions for Bialystok:

- Which computational examples would demonstrate value to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where by `computation` would immediately shorten real proofs?
- Which extraction targets matter first, if any?

Key Points:

- Formalized Mathematics gives Mizar something rare: a scholarly, citable publication layer over a formal library.
- But article identity and library organization are tightly coupled: refactoring the library strains published article structure, and package reuse has no journal-facing identity at all.
- Exposition wants narrative order; reuse wants dependency order. One structure cannot optimize both.

Formalized Mathematics theorem

- > library package (name, version)
- > module path and theorem FQN
- > statement fingerprint
- > verified artifact hash
- > origin id (e.g. mizar:GROUP_1:...)

Message:

- each layer answers a different question: scholarly citation, current location, semantic drift detection, reproducible verification, and historical continuity with MML and past Formalized Mathematics volumes.

9.3 - Who Gains What

Audience	Gain
readers	prose stays primary; formal source is one click away
maintainers	refactoring no longer rewrites published exposition
authors	articles cite stable identities, not file layouts
AI tools	prose for retrieval, fingerprints for exact context

Preserved:

- Formalized Mathematics remains a real journal with review and exposition;
- origin metadata keeps continuity with every existing MML citation.

Questions for Bialystok:

- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - retroactively, on revision, or not at all?

Source

```
-> TokenStream -> SurfaceAst -> ResolvedAst -> TypedAst  
-> CoreIr -> VcIr -> AtpProblem -> ProofCertificate  
-> VerifiedArtifact
```

Message:

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes - that is the entire point.

10.2 - Responsibility Split

Layer	Responsibility
frontend	lexing, parsing, recovery
resolver	imports, names, labels, namespaces
checker	soft types, clusters, registrations, overloads
elaborator	core logical representation
VC generator	proof and algorithm obligations
ATP layer plus kernel	untrusted search, then acceptance by checking
artifact emitter	stable outputs for tools and dependents

10.3 - Where The Eight Stories Live

Story	Pipeline home
dependencies	resolver, package manager
structures and automation	checker (inheritance and cluster graphs, traces)
search vs trust	ATP layer, kernel, certificates
scale	artifacts, fingerprints, scheduler
templates	elaborator (checked instantiation)
algorithms	VC generator, MVM
publication	artifact emitter, doc generation

Message:

- the stories are not eight separate projects; they are one pipeline seen from eight user-visible pains.

Key Phrase:

```
Reject what must not pass  
before  
Accept everything that should pass
```

Key Points:

- soundness bugs outrank parser gaps; kernel-adjacent tests emphasize malformed and failing evidence first;
- accepted-language coverage grows behind that shield.

Phases:

- ① End of 2026, alpha: frontend and parser for a core subset, import and module resolution prototype, structured diagnostics, early artifacts.
- ② 2027, migration laboratory: 3-5 representative MML articles translated by hand and by script; every mismatch recorded as a classified issue.
- ③ 2027-2028, expansion: foundational set and relation fragments, functions and binary operations, algebraic structures, then dependency cones around successful fragments.

Non-goals for the alpha:

- full MML verification, final compatibility layer, stable AI protocol.

Key Points:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP-plus-certificate;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

11.3 - Compatibility Policy And Risks

Policy:

- preserve theorem identity through origin metadata;
- keep compatibility aliases where they help migration;
- record every divergence from old behavior with a reason and a test.

Risk	Mitigation
compatibility work consumes the project	representative slices first, no big-bang translation
registrations behave differently	trace artifacts and comparison reports early
package layout breaks journal links	origin metadata, article-to-library identifiers
AI edits hide migration mistakes	Red edits stay forbidden; verifier artifacts required

Key Points:

- a prioritized list of migration benchmark articles;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

Questions:

- Which MML articles are small but structurally representative?
- Which idioms are culturally essential, beyond technical convenience?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is serious?

Slide question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Final slide:

Mizar Evo should be modern where scale demands it,
and conservative where Mizar's mathematical identity depends on it.

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Structure definition	algstr_0.miz	37-40	Frames 0.2, 3.1
Article environment	algstr_0.miz	15-25	Frame 2.1
Registration/cluster	algstr_0.miz	104-109	Frame 4.1

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Proof citation	struct_0.miz	637-643	Frame 5.4
Induction scheme	nat_1.miz	near 90	Frame 7.1

Source URLs:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- NAT_1: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Backup B. Specification Reference Map (1/2)

The slides show examples only. The authoritative grammar and semantics live in the specification; this map replaces the EBNF that earlier drafts put on slides.

Topic	Specification source (under doc/spec/en/)
Modules and imports	12.modules_and_namespaces.md
Structures and inheritance	05.structures.md
Attributes and modes	06.attributes.md, 07.modes.md
Registrations and reductions	17.clusters_and_registrations.md
Templates and schemes	18.templates.md
Algorithms and MVM	20.algorithm_and_verification.md
Packages and artifacts	23.package_management_and_build_system.md

Backup B. Specification Reference Map (2/2)

Topic	Specification source (under doc/spec/en/)
Cross-chapter grammar	<code>appendix_a.grammar_summary.md</code>
Worked library sketches	<code>sample_codes.md</code>

Required diagrams for the final deck:

- 1 Three pressures on a proven design (Part 1).
- 2 Environment-to-import migration (story 1).
- 3 Structure inheritance and diamond coherence (story 2).
- 4 Reasoning boundary: semantics / ATP search / kernel checking (story 4).
- 5 Certificate object and replay (story 4).
- 6 Incremental fingerprint graph (story 5).
- 7 Formalized Mathematics article-to-library link model (story 8).

Required diagrams for the final deck:

- 1 Full pipeline with story overlay (Part 10).
- 2 Roadmap timeline (Part 11).

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

- 1 Introduction: why Mizar needs evolution now.
- 2 Mizar as baseline: readability, MML, Formalized Mathematics.
- 3 Design principles and the three pillars.
- 4 Language evolution: dependencies, structures, registrations, templates.
- 5 Verifier architecture, certificates, and the small kernel.
- 6 Verified computation and the MVM.
- 7 AI-safe proof development.

Possible sections:

- 1 Package-based library and publication workflow.
- 2 Migration plan and evaluation metrics.
- 3 Related work and collaboration agenda.

Use this checklist before converting to Beamer:

- Does every story open with a real cost, not a feature announcement?
- Does every criticism acknowledge why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, specification example, or sketch)?
- Is every exact excerpt attributed with article and line numbers?
- Does every story end with questions the audience can actually answer?
- Are Red AI edits clearly forbidden?
- Are migration claims measurable?

Use this checklist before converting to Beamer:

- Is EBNF absent from all frames?